



北方民族大学

本科毕业论文（设计）

题目：基于 Vulkan 的实时光线追踪混合渲染器
的设计与实现

学院名称：计算机科学与工程学院

专 业：软件工程

学 生 姓 名：向晨

学 号：20221889

指导教师姓名：韩强

企业指导教师：

论文提交时间：

北方民族大学本科毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人郑重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：_____

2026 年 05 月 日

基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

摘要

在现代实时计算机图形学中，传统的纯光栅化渲染在处理全局光照、精确物理软阴影与复杂镜面反射时存在固有的物理局限性，而离线级别的纯路径追踪又难以满足交互式应用的实时帧率需求。针对这一“画质与算力”的矛盾，本文基于现代显式图形 API Vulkan 1.3，设计并实现了一套工业级的高性能实时光线追踪混合渲染引擎。

本文的核心工作包括三个方面。首先，针对 Vulkan 繁琐的显存状态同步痛点，设计了数据驱动的 RenderGraph 调度架构，通过有向无环图的拓扑分析实现了管线屏障的自动推导与资源生命周期管理。其次，构建了高效的混合光照管线，将几何光栅化与硬件光线追踪深度解耦。在延迟渲染 G-Buffer 的基础上，结合偏导数面法线偏移与 Ray Query 扩展实现了精准无伪影的软硬阴影，并利用 RT Pipeline 全面求解 Cook-Torrance 微面元模型，完成了基于物理（PBR）的多次镜面反射计算。最后，针对 1 SPP 极低采样率带来的蒙特卡洛高频噪声，完整集成了时空方差导向滤波（SVGF）降噪系统。通过几何运动矢量的时域历史重投影与动态方差引导的多轮 A-Trous 空域滤波，成功实现了高保真度的抗噪平滑处理。

性能测试与效果分析表明，本系统在 2K 原生分辨率下，能够以实时的性能开销（稳定大于 60 帧）输出平滑且物理正确的照片级渲染画面。本文的研究验证了以 RenderGraph 为底座、结合光栅化、硬件光追与 SVGF 降噪的混合架构在现代底层图形引擎开发中的高效性与工程可行性。

关键词: Vulkan; 实时光线追踪; 混合渲染; RenderGraph; 基于物理的渲染 (PBR); SVGF 降噪

Design and Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

ABSTRACT

In modern real-time computer graphics, traditional pure rasterization rendering has inherent physical limitations when dealing with global illumination, precise physical soft shadows, and complex specular reflections, while offline-level pure path tracing is difficult to meet the real-time frame rate requirements of interactive applications. To address this contradiction between "image quality and computing power", this paper designs and implements an industrial-grade high-performance real-time ray tracing hybrid rendering engine based on the modern explicit graphics API Vulkan 1.3.

The core work of this paper includes three aspects. First, a data-driven RenderGraph scheduling architecture was designed to address the tedious GPU memory state synchronization pain points of Vulkan. Through topological analysis of a directed acyclic graph, the automatic derivation of pipeline barriers and resource life-cycle management were achieved. Second, an efficient hybrid lighting pipeline was constructed to deeply decouple geometric rasterization from hardware ray tracing. Based on deferred rendering G-Buffer, combined with derivative face normal offset and Ray Query extension, precise and artifact-free soft and hard shadows were implemented. Furthermore, the RT Pipeline was used to comprehensively solve the Cook-Torrance microfacet model, completing multi-bounce specular reflection calculations based on physics (PBR). Finally, addressing the high-frequency Monte Carlo noise caused by the extremely low sampling rate of 1 SPP, a complete Spatiotemporal Variance-Guided Filtering (SVGF) denoising system was integrated. High-fidelity anti-noise smoothing was successfully achieved through temporal history reprojection of geometric motion vectors and multi-round spatial A-Trous filtering guided by dynamic variance.

Performance tests and effectiveness analysis show that this system can output smooth and physically correct photo-realistic rendering results with real-time performance overhead (stable above 60 FPS) at 2K native resolution. This research verifies the efficiency and engineering

feasibility of a hybrid architecture based on RenderGraph, combining rasterization, hardware ray tracing, and SVGF denoising in modern low-level graphics engine development.

Key words: Vulkan; Real-time Ray Tracing; Hybrid Rendering; RenderGraph; Physically Based Rendering (PBR); SVGF Denoising

目 录

摘要	II
ABSTRACT	III
第一章 绪论	1
1.1 研究背景及意义	1
1.2 研究现状	2
1.3 本文研究内容	3
1.4 论文总体结构	3
1.5 本章小结	3
第二章 实时混合渲染相关技术	4
2.1 实时混合渲染技术	4
2.1.1 光栅化	4
2.1.2 光线追踪	4
2.2 基于物理的渲染	5
2.3 实时渲染管线	5
2.3.1 前向渲染管线	5
2.3.2 延迟渲染管线	5
2.4 图形 API 选型分析	6
2.4.1 OpenGL 及其他 API 概述	6
2.4.2 Vulkan 的核心优势	6
2.5 本章小结	7
第三章 需求分析	8
3.1 可行性分析	8
3.1.1 经济可行性	8
3.1.2 技术可行性	8
3.1.3 操作可行性	8
3.2 系统业务流程分析	8
3.3 系统功能性需求分析	9

3.3.1	用户功能需求	9
3.3.1.1	场景与资产管理模块	9
3.3.1.2	Debug UI 界面	10
3.3.1.3	场景交互与观察模块	10
3.4	系统非功能性需求分析	11
3.4.1	性能与响应时间	11
3.4.2	安全性与稳定性	12
3.4.3	可用性与可靠性	12
3.4.4	易用性	12
3.4.5	可维护性与可扩展性	12
3.5	系统核心类设计	13
3.5.1	全局管理与框架类设计	13
3.5.2	资源管理与场景组织类设计	13
3.5.3	渲染调度引擎核心类设计	14
3.6	本章小结	15
第四章	概要设计	16
4.1	引擎的架构设计	16
4.1.1	Walnut 框架架构分析	16
4.1.2	RenderGraph 调度架构分析	17
4.1.3	本引擎架构设计	17
4.2	功能模块划分	18
4.2.1	平台适配模块	18
4.2.2	核心框架模块	18
4.2.3	资源管理模块	19
4.2.4	渲染调度模块	19
4.2.5	交互调试模块	19
4.3	混合渲染管线流程设计	19
4.3.1	第一阶段：几何特征采集	20
4.3.2	第二阶段：物理光影计算	20
4.3.3	第三阶段：时空信号重建	21
4.3.4	第四阶段：光照合成与后期处理	21
4.4	本章小结	21
第五章	详细设计	22
5.1	平台适配模块详细设计	22

5.1.1	基于 volk 的动态函数指针加载机制	23
5.1.2	硬件寻址与特性链 (Feature Chaining) 注入	24
5.1.3	交换链 (Swapchain) 管理与异常恢复	24
5.1.4	验证层与运行时调试机制	24
5.2	核心框架模块详细设计	24
5.2.1	显存池化管理与亚分配 (Sub-allocation) 策略	24
5.2.2	细粒度异步任务调度系统	25
5.2.3	应用程序生命周期与分层架构	25
5.3	资源管理模块详细设计	26
5.3.1	基于硬件特性的两级加速结构 (AS) 维护	27
5.4	渲染调度模块详细设计	28
5.4.1	Setup 阶段: 虚拟资源与依赖声明	30
5.4.2	Compile 阶段: 拓扑排序与显存复用算法	30
5.4.3	Execute 阶段: 屏障推导与异步执行	30
5.5	混合渲染管线详细设计	30
5.5.1	基于延迟光栅化的 G-Buffer 预处理	31
5.5.2	硬件加速光线追踪与采样算法	31
5.5.3	时空域联合降噪 (SVGF) 算法设计	32
5.5.4	延迟光照解耦与最终合成	32
5.6	交互调试模块设计	32
5.7	本章小结	33
第六章	系统实现与测试	34
6.1	系统开发与运行环境	34
6.2	核心功能模块实现要点	34
6.2.1	基于 RAII 与延迟队列的资源管理实现	34
6.2.2	混合渲染管线的通道化压缩实现	35
6.3	系统测试	36
6.3.1	功能性测试用例	36
6.4	渲染效果对比与质量评估	37
6.4.1	SVGF 降噪效果分析	37
6.4.2	TAA 抗锯齿效果评估	37
6.5	性能定量评估与分析	37
6.5.1	各阶段 GPU 耗时统计	37
6.6	本章小结	38

第七章 总结与展望	39
7.1 全文工作总结	39
7.2 研究不足与未来展望	39

第一章 绪论

1.1 研究背景及意义

实时渲染是聚焦于制作和分析实时图像的计算机图形学的子领域，其聚焦于在极短的时间内完成从三维几何数据到二维图像的计算与生成。在典型的实时交互应用中，系统需要持续捕获用户的输入，更新虚拟环境状态，并以较高的刷新率输出连续的图像序列。一般大于 30FPS 的渲染被称为实时渲染，10FPS 左右的我们称为可交互渲染，再慢的就是离线渲染。实时渲染技术目前广泛应用于强交互的 3D 游戏和带 3D 的应用中，例如，游戏，VR/AR，虚拟仿真等，有巨大的应用前景和商业价值。

在技术路线上，实时渲染主要分为光栅化（Rasterization）与光线追踪（Ray Tracing）两种基础架构。光栅化技术基于多边形投影原理，其算法流程高度契合 GPU 的单指令多数据流（SIMD）并行计算架构，能够以极高的效率处理大规模几何数据。然而，光栅化本质上是一种基于局部信息的计算模型，缺乏对场景全局几何拓扑的直接感知能力。因此，在处理全局光照（Global Illumination）、软阴影及物理反射时，传统光栅化管线主要依赖屏幕空间近似算法（如屏幕空间反射 SSR、屏幕空间环境光遮蔽 SSAO）或预计算技术（如阴影贴图、光照探针）。这些启发式近似方法在处理复杂遮挡关系与动态场景时，往往会产生走样（Aliasing）及物理失真问题。

相对而言，光线追踪技术通过在三维空间中模拟光线的物理传播、散射与折射路径，能够更准确地求解渲染方程，从而生成具备物理真实感的光影效果。但由于光线在场景中弹射路径具有较高的随机性，导致内存访问呈现非连续特征，难以充分利用 GPU 的并行算力，因此长期被限制在离线渲染领域。

随着 GPU 硬件算力的提升以及专用硬件光线追踪加速单元（如 RT Core）的引入，实时光线追踪技术逐渐具备了工程可行性。为平衡渲染质量与计算开销，混合渲染架构（Hybrid Rendering Architecture）成为当前的主流发展方向。该架构将光栅化在初级可见性（Primary Visibility）计算上的高效率，与光线追踪在次级光线（如反射、阴影）求解上的准确性相结合，有效弥补了单一管线的不足，是目前图形学界重点研究的核心课题。

1.2 研究现状

图形渲染管线及底层硬件经历了长期的演进过程。20 世纪 90 年代末，图形硬件主要采用固定渲染管线（Fixed-Function Pipeline），开发者仅能通过配置预设状态来控制渲染输出。2001 年起，可编程着色器（Programmable Shaders）的引入使开发者能够自定义顶点变换与像素着色逻辑。至 2006 年，统一着色器架构（Unified Shader Architecture）的提出，打破了不同着色阶段计算资源的物理隔离，为现代通用 GPU 计算奠定了基础。2018 年是实时渲染的分水岭，Microsoft 发布了 DXR（DirectX Raytracing）框架，并且 NVIDIA 在同年推出了基于 Turing 架构的 RTX 平台，让实时光线追踪成为可能。

在底层图形应用程序接口（Graphics API）方面，传统 API（如 OpenGL 与 DirectX 11）由于设计于单核 CPU 时代，采用了高度封装的全局状态机模型。在这种架构下，内存管理与线程同步操作由图形驱动程序隐式处理，导致在处理复杂场景时产生较高的 CPU 驱动开销，限制了多核处理器的并发能力。为此，工业界推动了向显式图形 API（Explicit Graphics API）的范式转移，如 Vulkan 与 DirectX 12。此类 API 将显存的分配与释放、多线程命令缓冲区的录制以及管线同步屏障（Pipeline Barriers）的控制权显式移交至应用层，显著降低了驱动开销，提升了硬件算力的利用率。

渲染器是游戏引擎中核心的子系统之一，它负责图形绘制，接收渲染命令和资源数据（顶点、纹理、Shader 等），并调用 OpenGL/D3D/Vulkan 等图形 API，将游戏世界中的数据（模型、纹理、光照、摄像机等）转换为最终呈现在屏幕上的图像。由 ID Software 开发的游戏《德军总部 3D（Wolfenstein 3D）》《毁灭战士（DOOM）》取得了巨大成功，其主要的开发者约翰卡马克把游戏开发可以重复使用的代码提取出来，创造了游戏引擎这一概念。DOOM 引擎是第一个用于商业授权的引擎，后来开发的 Quake 引擎是真正的 3D 引擎，人们普遍认为游戏《雷神之锤》带来了独立 3D 显卡的革命。如今的商业游戏引擎市场基本上是 Unreal 和 Unity 两巨头，在开源领域也有 Orge，Godot 等引擎，当然也还有专门的渲染引擎，比如 V-Ray，OctaneRender 等，给许多引擎和渲染开发者提供了优秀的学习资料，促进了游戏引擎，渲染领域的发展。

在渲染引擎架构层面，随着渲染管线复杂度的增加，传统的面向对象绘制指令流逐渐难以维护。现代工业级渲染引擎广泛引入了基于数据驱动的渲染图架构 Render-Graph。该架构通过构建有向无环图（DAG），在编译期自动推导各渲染通道（Render Pass）之间的数据依赖关系，从而实现显存的动态复用与同步屏障的自动插入，是目前处理复杂混合渲染管线状态调度的标准方案。

此外，在实时光线追踪的工程实践中，受限于实时计算的性能预算，系统通常仅能采用极低的采样率（如单像素单样本，1-SPP），导致原始渲染结果存在严重的蒙特卡洛噪声。因此，时空重建与降噪技术（Spatiotemporal Reconstruction and Denoising）成为当前的研究热点。以时空方差引导滤波（SVGF）为代表的算法，通过提取历史帧的时间相关性信息并结合当前帧的空间几何特征，能够有效平滑高频噪声。降噪算法的执行效率与重建质量，已成为决定混合渲染管线最终视觉表现的关键因素。

1.3 本文研究内容

本文基于 Vulkan 1.3 规范设计并实现了一个三维实时混合渲染引擎，主要研究内容包括以下几个方面。

1. 研究 Vulkan 1.3 渲染管线，利用动态渲染等新特性优化引擎底层架构，探索资源管理流程。
2. 研究已有渲染引擎架构，从而设计本引擎的架构和模块。
3. 构建混合渲染管线，结合光栅化的高效几何处理能力与光线追踪的物理精确照明，实现高质量的软阴影与反射效果。

1.4 论文总体结构

以下是对本论文各个章节的概要：

- **第一章绪论：**介绍实时混合渲染的研究背景、研究现状，并简述本文的主要工作。
- **第二章理论基础：**深入分析 PBR 物理模型、Vulkan 渲染机制及硬件光追原理。
- **第三章需求分析：**从可行性、业务流程、功能与非功能指标等方面详细阐述引擎需求。
- **第四章概要设计：**论述系统分层架构、功能模块划分及显存逻辑表结构设计。
- **第五章详细设计：**以需求为基础，对渲染图算法、光追逻辑及时空重建模型进行细致设计。
- **第六章系统实现与测试：**系统展示核心代码实现，通过功能测试用例与性能数据验证系统有效性。
- **第七章总结与展望：**总结全文工作并指出未来改进方向。

1.5 本章小结

本章阐述了混合渲染引擎的研究背景与意义，梳理了国内外图形技术的研究现状，并明确了本文的主要研究内容。最后给出了论文的整体章节安排，为后续内容的展开奠定了基础。

第二章 实时混合渲染相关技术

本章将对构建实时混合渲染器所涉及的核心技术进行系统性介绍，包括基础渲染技术、基于物理的渲染理论、实时渲染管线架构以及主流图形 API 的对比与分析。

2.1 实时混合渲染技术

渲染是指通过软件算法将包含几何、视点、纹理和照明等信息的三维（或二维）模型最终转换为二维图像的技术。这一过程可以类比为相机拍照，本质上是对人眼的模拟，并将三维空间信息投射到二维平面。渲染的数据源主要包含模型几何、相机参数和光源属性。针对这些数据的处理，目前主要存在两种主流算法路径。

2.1.1 光栅化

光栅化（Rasterization）是现代实时渲染中最经典、应用最广泛的技术之一，其核心思想是“将几何体投影至屏幕”。在光栅化流程中，场景中的几何体统一使用三角形网格表示。在经过顶点着色器处理及一系列矩阵变换（详见附录 A）后，三维模型被投射到屏幕裁剪空间。系统随后计算出每个三角形面投影到屏幕上对应的像素位置，并根据几何与材质信息计算该像素的最终颜色。

光栅化算法逻辑简单，其逐顶点变换与逐像素处理的特性极度契合 GPU 的并行计算架构。然而，光栅化在处理光照模型时进行了大量简化，光影效果相对单一，难以在不引入复杂经验公式的前提下模拟真实世界中复杂的全局光照现象。

2.1.2 光线追踪

与光栅化的“投射”思路不同，光线追踪（Ray Tracing）采用了一种逆向模拟光路物理传播的思路。它通过计算光线与物体的物理交互（如反射、折射、吸收），生成极具真实感的图像。光线追踪算法从虚拟相机出发，将屏幕视为视锥体的近平面。光线以相机为原点，穿过屏幕像素向三维场景中发射。当射线与场景中的空间加速结构（如层次包围盒 BVH）相交时，系统根据交点处的材质属性，递归地生成反射光线、折射光线以及阴影光线。通过累积各层光路的能量贡献，最终得到像素的物理精确颜色。

相较于光栅化，光线追踪在处理全局光照、镜面反射、透明折射及软阴影等细节时具有天然优势。但由于需要进行海量的射线-几何体求交运算，其计算成本极高。传统的纯光线追踪渲染难以满足高帧率实时交互的需求，通常用于对时效性要求较低的离线

渲染及影视特效领域。

2.2 基于物理的渲染

基于物理的渲染（Physically Based Rendering，简称 PBR）是指一系列基于物理原理建模的渲染技术集合。与早期通过经验公式（如 Blinn-Phong 模型）模拟高光的做法不同，PBR 旨在通过模拟光线在真实物理表面的分布与衰减规律，使画面表现出极高的真实感。

相较于传统技术，PBR 具有显著优势：其一，材质参数（如金属度、粗糙度）具有明确的物理意义，便于艺术资产的统一管理，降低了材质制作的复杂度；其二，光照表现具有高度的一致性，在不同光照环境下，物体材质始终符合能量守恒等物理规律，显著提升了场景的沉浸感。PBR 理论能够完美集成到混合渲染管线中，作为光影评估的核心准则。

2.3 实时渲染管线

在实时渲染领域，渲染管线定义了从场景数据到最终图像的完整计算流程。根据光照计算时机的不同，传统管线主要分为前向渲染管线与延迟渲染管线。

2.3.1 前向渲染管线

前向渲染（Forward Rendering）是最直观的管线模型。渲染器遍历场景中的每个物体，并针对物体上的每个可见像素，依次累加场景中所有光源对其产生的颜色贡献。该模型实现简单，能够天然支持透明混合与抗锯齿处理。但在光源数量较多的复杂场景下，由于无效的过度绘制（Overdraw），其计算开销会迅速增长。

2.3.2 延迟渲染管线

延迟渲染（Deferred Rendering）是现代高效渲染的主流技术，特别适用于多光源环境。该管线主要分为两个阶段：

1. **几何处理阶段：**渲染器不进行复杂光照计算，而是将像素的几何特征（位置、法线、反照率等）预先渲染到一系列纹理中，这些纹理集合被统称为 G-Buffer（Geometry Buffer）。
2. **光照处理阶段：**在 G-Buffer 的基础上，渲染器遍历光源，仅根据屏幕空间存储的信息进行着色计算。

该方法的优点是解耦了几何与光照，极大降低了光照计算开销；缺点在于 G-Buffer 会占用显著的显存带宽，且不利于直接处理透明物体。

2.4 图形 API 选型分析

图形应用程序接口（Graphics API）是连接软件逻辑与 GPU 硬件的桥梁。目前业界主流 API 包括 OpenGL、DirectX 和 Vulkan，它们共同构成了混合渲染器的工程基础。

2.4.1 OpenGL 及其他 API 概述

- **OpenGL:** 历史最悠久的跨平台 API，生态成熟且易于上手。但其由于底层状态机设计笨重，驱动开销巨大。自 2017 年发布 4.6 版本后，管理组织 Khronos Group 已转向 Vulkan 的开发，OpenGL 目前已停止重大更新。
- **DirectX:** 由微软开发，是 Windows 与 Xbox 平台的核心 API。其最新版本 DirectX 12 支持细粒度的硬件控制，在 3A 游戏开发中具有显著优势，但受限于特定的平台生态。
- **Metal:** 苹果公司推出的专用 API，优势在于极低的 CPU 提交开销及对苹果硬件的深度优化。

2.4.2 Vulkan 的核心优势

Vulkan 是新一代跨平台图形 API，彻底摒弃了传统状态机的设计，将内存管理、多线程同步及 API 验证等权责交由开发者控制。虽然这种设计增加了开发门槛，但换取了巨大的运行性能提升。在实时混合渲染中，Vulkan 通过成熟的光线追踪扩展（如 VK_KHR_ray_tracing 系列），为开发者提供了直接操控 RT Core 硬件单元的能力。图 2.1 简要展示了传统 API 与现代显式 API 的设计差异。

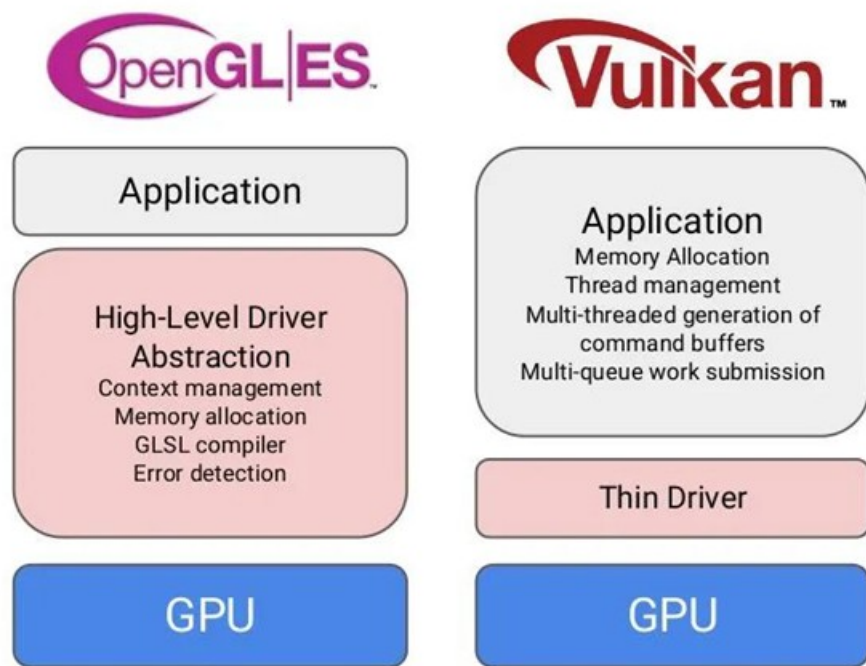


图 2.1 OpenGL 与 Vulkan API 架构设计对比图

2.5 本章小结

本章系统性地介绍了实时渲染的核心技术背景。从光栅化与光线追踪的原理对比出发，阐述了 PBR 材质理论的优越性，并详细剖析了不同渲染管线的执行逻辑。最后，通过对主流图形 API 的分析，确立了以 Vulkan 作为混合渲染器实现的底层基石。

第三章 需求分析

本章将从软件工程的角度对混合渲染引擎进行需求分析。通过对可行性、业务流程及功能/非功能需求的梳理，确立系统在图形表现、调度效率及用户交互方面的具体基准。

3.1 可行性分析

3.1.1 经济可行性

本项目完全基于开源生态构建，核心开发环境与依赖库均具有较高的经济效益：

- **开发成本：**Vulkan SDK、CMake、编译器（MSVC/GCC）以及图形驱动开发工具均可免费获取，无需支付商业许可证费用。
- **开源库集成：**集成了 GLFW、Dear ImGui、GLM 及 VMA 等开源领域流行的库，本身的开源证书限制较小，不要求支付版权费用。
- **硬件成本：**随着实时光线追踪显卡（RTX 系列）的普及，主流消费级硬件即可作为研发平台，研发设备投入处于合理区间。

3.1.2 技术可行性

- **标准支持：**系统基于 Vulkan 1.3 标准及 KHR Ray Tracing 扩展，利用 C++20 现代特性实现。当前硬件架构已提供成熟的硬加速求交单元（RT Core）。
- **知识获取：**Vulkan 的官方文档、开源引擎的源码以及学术论文为技术实现提供了丰富的参考资料。
- **开发工具：**目前的 Visual Studio, Visual Studio Code 等集成开发环境都能提供良好的开发环境，各种开源库也能支持引擎的资源加载，图像界面，输入输出等功能。利用 Vulkan 验证层（Validation Layers）及 RenderDoc 调试工具，方便调试。

3.1.3 操作可行性

- **交互直观性：**系统集成可视化 UI 调试面板，用户可实时通过 UI 界面，查看 Debug 信息，加载模型，移动光源，设置调试参数，以快速验证渲染的效果。
- **交互控制：**参考成熟的游戏引擎的相机，交互的实现，降低用户的操作门槛。

3.2 系统业务流程分析

本混合渲染器的业务流程如下所述：

1. **运行应用：**启动渲染器，初始化 Vulkan 上下文与图形驱动环境，加载窗口界面。
2. **场景加载与解析：**用户通过 UI 界面选择 3D 模型和天空盒，引擎会后台自动进行资源加载与解析。
3. **渲染节点构建：**系统根据用户选定的渲染路径（光栅化、混合路径或光追路径）自动构建 RenderGraph 拓扑图，分配资源和 Pass。
4. **交互调优：**用户通过菜单栏切换不同的渲染模块，实时调整光线方向，SVGF、TAA 开关，通过键盘鼠标调整相机位置和远近。
5. **数据反馈与可视化：**系统在主界面生成图像，同时可以选择中间视图，方便查看每个阶段的效果，以及耗时统计等 Debug 信息。

3.3 系统功能性需求分析

混合渲染器在设计时以图形开发者和环境艺术家为核心用户，基于其在场景搭建、效果调优及性能监控中的需求进行功能分析。以下 3.3.1 小节详细描述了用户的具体功能需求。

3.3.1 用户功能需求

用户作为系统的核心操作者，应具有如下功能：

3.3.1.1 场景与资产管理模块

该模块负责对渲染场景中的 3D 资产及其空间属性进行直观管理：

- **资源浏览与加载：**系统自动扫描指定目录下的模型与环境贴图。用户通过资源浏览器（Content Browser）可在多个 GLTF 模型间切换加载，并支持一键更换 HDR 环境天空盒。
- **场景层级管理：**用户通过层级面板（Hierarchy）实时查看当前场景中的所有实体对象，支持选中特定实例以进行后续操作，并提供安全移除选定实体的功能。
- **变换编辑：**支持对选中实体的平移（Position）、旋转（Rotation）和缩放（Scale）参数进行实时数值调节，并同步更新底层的硬件加速结构（AS）。

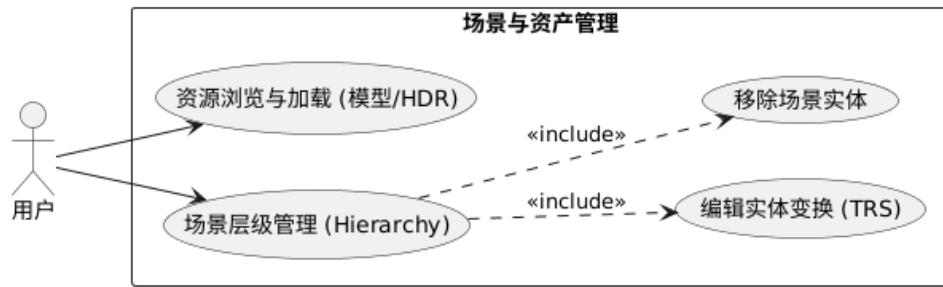


图 3.1 场景与资产管理用例图

3.3.1.2 Debug UI 界面

用户可以对渲染管线和环境光照进行深度定制，以达到最佳的画质表现：

- **渲染路径与视图切换：**用户可以在光栅化（Forward）、混合渲染（Hybrid）及路径追踪模式间实时切换。提供多种可视化模式，允许用户查看 Albedo、法线、材质参数、运动矢量以及 SVGF 方差图等中间附件。
- **渲染调试控制：**提供对光线追踪阴影、反射等独立控制的开关。针对信号重建，支持对 SVGF 时空降噪，TAA 抗锯齿等开关以查看效果。。
- **光源控制：**支持对主光源的方向、颜色进行调节，加载环境光照。
- **Debug 信息查看：**提供相机参数，帧数统计，性能分析等信息的实时显示，显示每个 Pass 的耗时，以及渲染管线拓扑图的的导出。

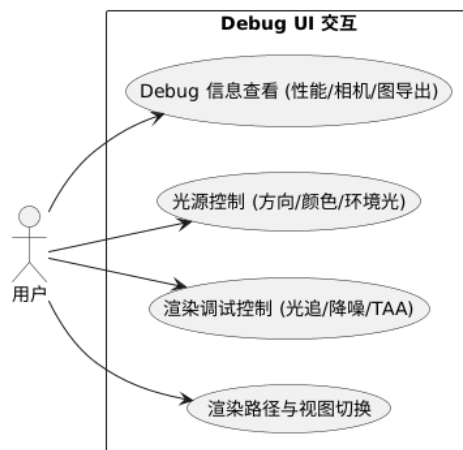


图 3.2 渲染控制与参数配置用例图

3.3.1.3 场景交互与观察模块

该模块为用户提供灵活的场景巡检手段与实时的系统状态反馈：

- **复合相机交互：**系统实现了一套基于弧球（Arcball）逻辑的编辑器相机。用户可通过“键盘 + 鼠标”的组合键模式进行操作：按住 Alt 键配合鼠标左键实现视角的环绕旋转，配合中键实现焦点的平移，配合右键或滚轮实现焦距缩放。该模式确保了用户能以任意角度观察场景细节。
- **交互状态反馈：**Debug UI 面板实时同步并显示相机的物理参数，包括相机的世界坐标（Position）、观察焦点（Focal Point）以及视场角（FOV），方便用户在调试特定渲染效果时精确记录机位。
- **系统性能监测：**集成实时的性能分析器，显示当前帧耗时（Frame Time）与帧率（FPS）波动。通过内置的时间戳查询技术，系统能够将渲染管线的内部执行细节可视化，允许用户监控每个渲染 Pass 的 GPU 执行开销及占比。

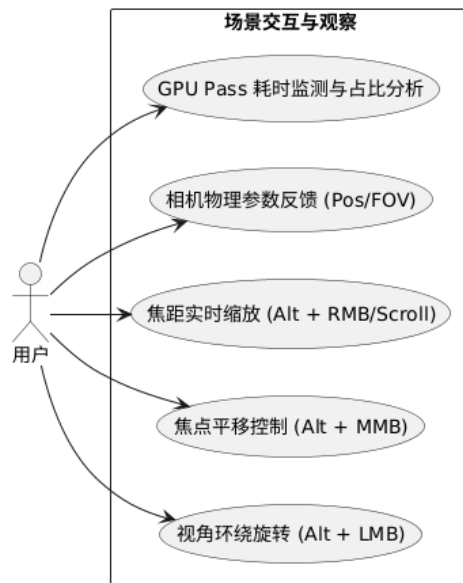


图 3.3 场景交互与观察用例图

3.4 系统非功能性需求分析

非功能性需求定义了系统在执行功能时所必须遵循的质量约束与技术指标。针对实时混合渲染系统的特性，本项目从性能、安全性、可用性、易用性以及可维护扩展性五个维度确立了非功能需求基准。

3.4.1 性能与响应时间

实时渲染系统对计算效率有着严苛的要求，需确保算法逻辑与数据流转的高度优化：

- **交互实时性指标：**在目标硬件平台上，系统应能维持稳定的亚秒级反馈。单帧渲染的端到端延迟应满足实时渲染的工业标准，确保在复杂场景漫游与参数调节时具有平滑的交互响应。
- **算法复杂度控制：**系统需针对计算密集型任务（如时空信号降噪）进行时间与空间复杂度的深度优化。利用并行计算等技术规避性能瓶颈，确保系统性能不随场景复杂度的增长而出现非线性骤降。

3.4.2 安全性与稳定性

在底层图形 API 驱动下，渲染系统必须构建坚实的资源保护与执行同步机制：

- **底层资源安全：**系统需实现一套基于引用计数与延迟销毁的显存管理机制。通过 RenderGraph 的资源生存区间分析，确保 GPU 正在引用的缓冲区或图像资源不会被非法释放，从架构层面预防驱动层面的访问冲突。
- **执行同步健壮性：**系统需要规范管理异步流水线中的资源依赖。通过自动化同步引擎推导并生成最优的同步屏障，避免由于读写竞态导致的画面闪烁或硬件超时重置。

3.4.3 可用性与可靠性

系统应具备在复杂运行环境下保持稳定输出的能力，并提供有效的容错逻辑：

- **状态切换可靠性：**在动态切换渲染路径（如从光栅化切至光线追踪）或调整配置参数时，系统需具备健壮的状态机管理能力，确保管线重建过程中的显存状态转换合法，不发生崩溃。
- **异常监控与诊断：**系统需要有良好的监测系统，能够实时捕获硬件特性缺失（如不支持 RT 扩展）或着色器编译异常，并向开发者提供详细的底层诊断信息。

3.4.4 易用性

系统界面设计与交互逻辑应符合图形行业规范，降低操作者的认知负担：

- **交互直觉性：**参数调节面板与场景管理接口的设计应遵循数字内容创作工具的通用逻辑。常用渲染配置应提供直观的可视化调节手段，实现“所见即所得”的操作体验。
- **辅助支持能力：**系统需配备完善的实时日志监测与诊断工具，并提供符合行业标准的摄像机控制系统，确保用户能够低成本、高效率地进行场景观察与算法验证。

3.4.5 可维护性与可扩展性

渲染系统的架构设计应支持长期的功能迭代与模块替换：

- **高度模块化解耦：**系统需采用逻辑层与驱动层分离的架构。通过高度抽象的调度中枢，使得新的渲染算法节点能够以插件化的方式接入，而无需对核心同步框架进行破坏性修改。
- **架构透明度：**系统需维持清晰的代码结构，确保其内部的数据流向与组件协作关系易于理解，从而降低二次开发与性能调优过程中的维护成本。

3.5 系统核心类设计

本节通过一系列类图展示系统的静态设计结构，涵盖从全局框架到具体渲染调度的协作关系。系统采用高度模块化的 C++ 设计，其核心类协作主要分布在框架管理、资源调度、场景组织及渲染执行四个主要维度。

3.5.1 全局管理与框架类设计

系统以 Application 为单例核心，通过组合模式聚合 Window 和 VulkanContext 确立运行环境。分层框架利用 `std::vector<std::shared_ptr<Layer>>` 构建逻辑栈，支持 EditorLayer 等功能层级的一致性分发。其中 VulkanContext 对 VulkanInstance 与 VulkanDevice 进行了深度封装，为全系统提供统一的硬件能力抽象。

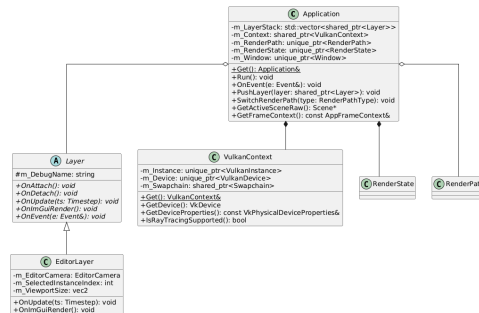


图 3.4 全局管理与分层框架类图

3.5.2 资源管理与场景组织类设计

系统采用组件化（Component-based）的场景组织结构。ResourceManager 负责 Vulkan 资源的 RAII 封装，通过基于线程池（Thread Pool）的多线程后台处理流水线，实现资产加载与逻辑更新的并行化，确保了大规模场景导入时的系统响应效率。Scene 类通过 Entity 容器组织场景对象，每个实体由 TransformComponent（TRS 变换）与 MeshComponent（模型引用）构成。系统通过对这些组件的实时追踪，自动维护并同步硬件加速结构（AS）。

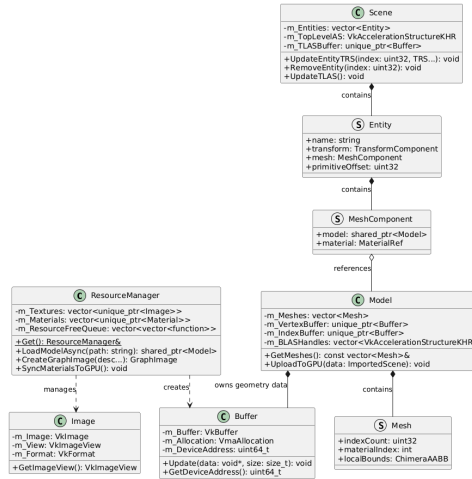


图 3.5 资源管理与场景组织类图

3.5.3 渲染调度引擎核心类设计

渲染调度由声明式的 RenderGraph 引擎驱动。系统通过 PassBuilder 捕获各渲染通道对显存资源的读写（Read/Write）依赖，从而推导出资源的物理生存区间。在执行阶段，引擎为不同类型的通道分发对应的执行上下文（ExecutionContext），屏蔽了底层同步屏障与镜像布局转换的复杂性，实现了高层次的架构解耦。

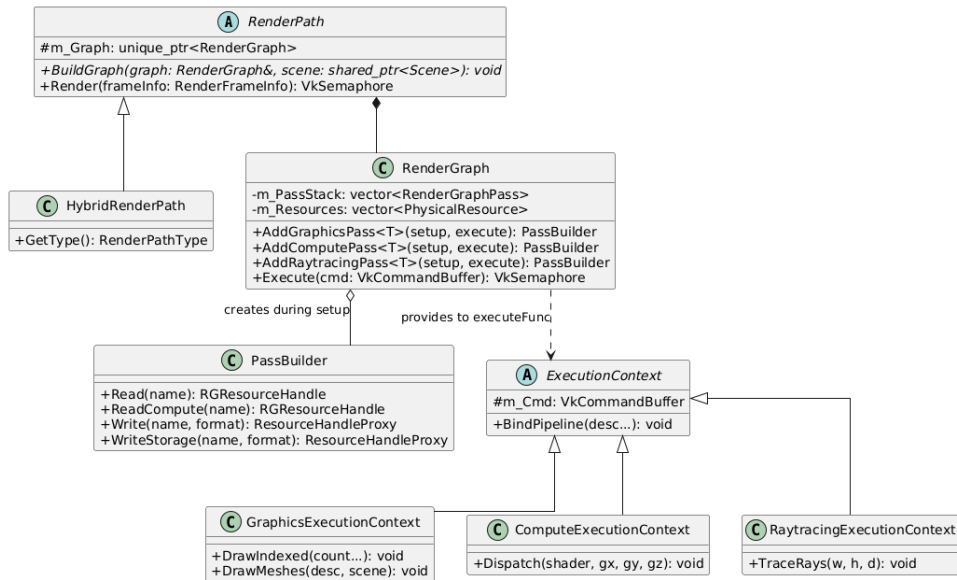


图 3.6 RenderGraph 调度引擎类图

3.6 本章小结

本章通过对可行性、业务流程及功能模块的深度拆解，明确了本渲染引擎混合渲染引擎的技术基准与交互标准。通过用例分析确立了用户在场景管理与管线调优中的核心需求，并通过详细的类图设计勾勒出了系统各子模块间的协作逻辑。本章的研究成果为下一章进入概要架构设计提供了明确的功能指导与工程依据。

第四章 概要设计

本章将从软件工程的视角出发，对混合渲染引擎进行概要设计。从参考引擎和优秀设计出发，借鉴并设计出本引擎的架构与模块。

4.1 引擎的架构设计

随着多年的发展，目前有许多成熟的引擎可供参考。本项目在底层架构上参考了结构清晰的 Walnut 框架，而在核心图形调度上则引入了现代化的 RenderGraph 系统。本节将对这两种核心技术架构进行理论分析，并引出其在本项目中的设计。

4.1.1 Walnut 框架架构分析

Walnut 是从游戏引擎 Hazel 中抽象出来的微内核框架，它剔除了游戏引擎中繁重的物理、音频、脚本等模块，仅保留了最核心的渲染循环与窗口管理逻辑。

Walnut 使用了微内核与层叠式架构（Layer Stack）。如图 4.1 所示，整个框架被严格划分为应用外壳、逻辑层栈和平台抽象层。其关键机制在于确立了基于生命周期回调函数（Lifecycle Callbacks，如重载 `OnUpdate`、`OnRender`、`OnImGuiRender` 方法）的渲染管线切入与事件分发系统。框架在每一帧通过自底向上的遍历，依次触发各逻辑层的回调，实现了渲染画面与交互调试 UI 的分离，也让本项目在开发混合渲染管线时，无需关注底层的窗口重建与系统级事件路由细节。

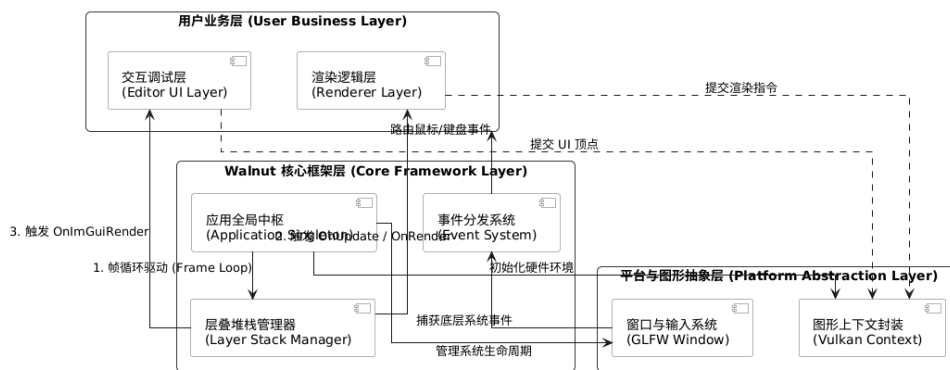


图 4.1 Walnut 框架组件架构图

4.1.2 RenderGraph 调度架构分析

RenderGraph, 也可以叫 *Frame Graph*, 最早由 EA 寒霜引擎 (Frostbite Engine) 团队在 GDC 2017 大会上提出。*Frame Graph* 由 RenderPass 和 Resource 构成。RenderPass 是一份元数据, 定义了一个完整的渲染流程需要用到的所有资源。Resource 是指 RenderPass 使用的 Texture、Shader 等资源。每个 RenderPass 都会指定对应的 Input 和 Output 资源, 这样 RenderPass 和 Resource 就形成了有向非循环图 (DAG) 结构。由于 RenderGraph 可以获得一帧的所有信息, 所以也被称为 *Frame Graph*。

使用该系统能够在执行任何实际的 GPU 绘制指令前, 纵观一帧的全局依赖信息, 从而自动推导出最优的 Pass 执行顺序、精准插入底层同步屏障 (Barriers), 并实现显存级别的高效复用。

4.1.3 本引擎架构设计

结合上述对 Walnut 架构与 RenderGraph 调度架构的理论分析, 本项目使用了图 4.2 中的五层拓扑架构体系:

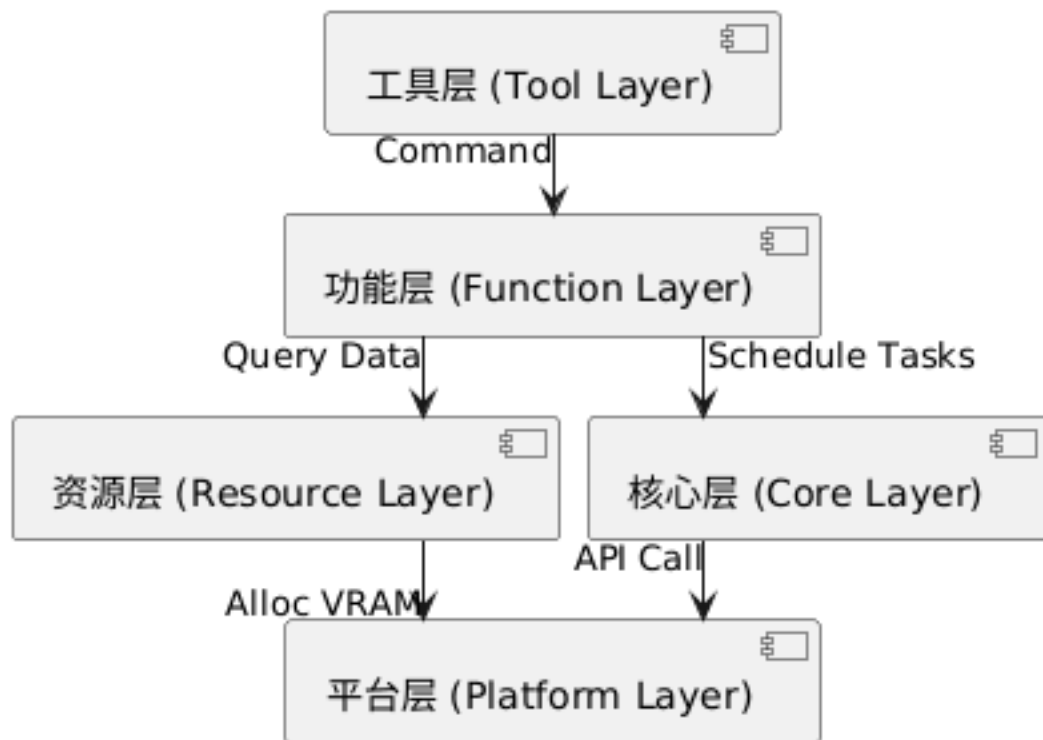


图 4.2 引擎五层分层拓扑架构图

1. 平台层 (Platform Layer): 位于架构最底层, 引擎的硬件抽象层 (HAL)。其核心

职责是定义系统与底层图形 API 及操作系统之间的通信边界，该层参考了 Walnut 框架的平台抽象思想，确保上层逻辑不受特定显卡硬件或系统驱动细节的影响。

2. **核心层 (Core Layer):** 提供全局基础设施服务。该层专注于为全引擎提供底层的任务并行调度、显存/内存原始分配策略以及基础数学运算支持，是整个系统运行的基石。
3. **资源层 (Resource Layer):** 负责资产的生命周期管理与数据组织。其职责是建立一套标准的数据加载与存储规范，将磁盘上的原始资源（模型、贴图等）转化为引擎可高效访问的内存对象，并负责动态维护光线追踪所需的顶层/底层硬件加速结构（TLAS/BLAS）。
4. **功能层 (Function Layer):** 引擎的渲染算法驱动核心。该层承载了 RenderGraph 调度系统，负责协调资源层的数据与核心层的计算能力，实现具体的混合渲染管线调度、驱动渲染管线，将抽象的渲染图元转化为真实的物理光影。
5. **工具层 (Tool Layer):** 位于架构顶端，直接面向用户。其通过调用功能层提供的接口，实现可视化的资源加载，交互调试、性能监控以及渲染参数设置等功能。

4.2 功能模块划分

在确立了引擎的五层拓扑架构后，本节将对该系统进行具体的功能化模块拆解。

4.2.1 平台适配模块

平台适配模块对应架构最底部的“平台层”，主要承担抹平物理硬件与操作系统差异的核心职责。在功能规划上，该模块涵盖三大维度的管理：首先是图形硬件上下文的建立，负责全盘接管物理显卡算力评估、逻辑环境初始化及呈现画布的创建；其次是系统级事件的路由机制，负责捕获底层操作系统的窗口变换与外设输入，并安全下发至引擎中枢；最后是标准硬件接口的抽象，其根本目的在于为上层的渲染逻辑提供一个高度兼容、完全透明的底层图形驱动环境，防止底层状态机反向污染业务代码。

4.2.2 核心框架模块

核心框架模块对应五层架构中的“核心层”，是维系全引擎运转的系统级中枢。该模块的职责聚焦于基础计算资源的统筹与调度。第一项核心功能是全局生命周期与层叠堆栈（Layer Stack）的管理，负责稳定驱动引擎的渲染主循环；第二项功能是内存与显存的分配策略管控，通过设计科学的内存池化机制，从根本上阻断高频渲染引发的碎片化灾难；第三项功能则是构建高吞吐量的多线程异步任务分发系统，确保诸如资源解析等计算密集型重负载能够无缝转移至后台，从而最大化保障主渲染线程的指令提交效

率。

4.2.3 资源管理模块

资源管理模块对应架构中的“资源层”，专职负责海量三维资产的摄入、流转与空间组织。针对混合渲染的严苛数据需求，该模块的核心功能包括：构建非阻塞式的资产加载流水线，将磁盘中的标准拓扑网格与物理级材质（PBR）转化为内存中可被硬件极速访问的标准化对象；实现场景实体（Entity）与组件逻辑的彻底剥离。此外，该模块还肩负着一项关键使命——实时追踪场景内实体的空间位移，并据此动态维护光线追踪所需的三维空间加速结构（如层次包围盒），确保底层的求交算法能够时刻获取绝对一致的物理世界状态。

4.2.4 渲染调度模块

渲染调度模块对应架构中的“功能层”，是本题“数据驱动渲染”思想的具体功能落实点。该模块的功能完全围绕“渲染任务图（RenderGraph）”展开：对外，它向业务层提供声明式的管线注册接口，收集每帧复杂的资源依赖意图；对内，它在每一帧指令提交前执行严格的图论拓扑分析与合法性校验。通过宏观审视全局依赖，该模块能够自动化推导出最优的通道（Pass）执行序列、精准生成避免数据冒险的同步屏障，并实现生命周期不重叠资源的显存复用。它极大降低了混合管线的工程复杂度，是整个渲染系统的大脑。

4.2.5 交互调试模块

交互调试模块对应架构顶端的“工具层”，旨在为引擎开发者提供一个非侵入式、高保真辅助分析与控制环境。其功能设计划分为两大核心维度：一是图形算法的“实时反控”，模块通过数据绑定机制，允许用户在前端面板动态调整诸如光线采样数、降噪时空权重等核心管线参数，并即刻反馈于最终画面，实现参数的闭环调优；二是物理硬件级的“性能侦测”，通过在渲染流水线的关键阶段埋点计时，实时捕捉并可视化输出 GPU 计算节点的微秒级耗时明细，为混合管线的算力瓶颈定位提供最权威的数据支撑。

4.3 混合渲染管线流程设计

在明确了引擎的模块化分工后，本节将深入探讨由渲染调度模块驱动的核心业务逻辑——混合渲染管线（Hybrid Rendering Pipeline）。该管线的设计目标是打破传统前向渲染（Forward Rendering）算力浪费的局限，构建一条兼顾执行效率与物理真实感的数据流转通道。

整个混合管线在宏观架构上被严格解耦为四个依次递进的概念阶段（如图 4.3 所

示），前一阶段的输出将作为后一阶段的输入先验条件。

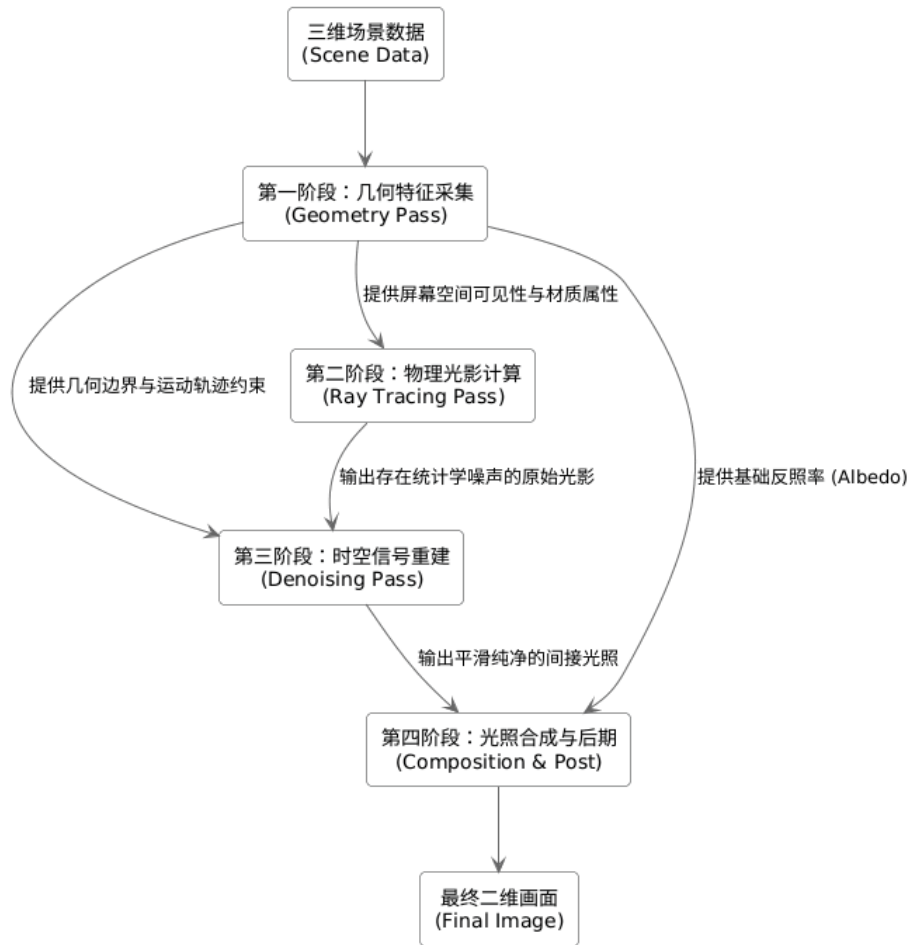


图 4.3 混合渲染管线宏观概念流转图

4.3.1 第一阶段：几何特征采集

混合管线的首个环节是建立对当前视角的“屏幕空间几何认知”。系统在此阶段不进行任何复杂的光照解算，而是专职负责剥离场景的几何体信息与材质属性。通过将屏幕可见元素的深度、法线、反照率及运动轨迹等特征分离存储，管线成功将“几何处理”与“光照计算”在时间轴上彻底解耦，为后续阶段避免了严重的像素过度绘制（Overdraw）开销。

4.3.2 第二阶段：物理光影计算

在获取了屏幕空间的可见性信息后，管线进入光影解算核心。该阶段利用几何采集阶段提供的表面特征，在三维空间中向光源及周围环境发射射线，以模拟真实世界中光线的遮蔽、反射与衰减行为。这种基于物理的计算方式虽然能获得高度精确的光照分布，

但受限于实时渲染的算力预算，本阶段输出的初步光影信号通常伴有强烈的随机噪声。

4.3.3 第三阶段：时空信号重建

为了弥补第二阶段低采样率带来的画面瑕疵，管线引入了专用的信号重建（降噪）阶段。该阶段充分复用第一阶段提取的几何边界与时域运动轨迹，通过跨帧的历史数据累积与当前帧的空间滤波，对原始噪声信号进行数学平滑处理。其宏观目的在于以极低的后处理性能开销，还原出等效于高采样率下的纯净光照结果。

4.3.4 第四阶段：光照合成与后期处理

在获取了纯净的各项光照分量后，管线进入最终闭环。合成模块将反照率等基础颜色与重构后的物理光影进行数学合并，还原出最终的材质质感。最后，画面将流经后期处理矩阵，完成高动态范围（HDR）到低动态范围（LDR）的色彩映射与抗锯齿处理，输出供显示设备呈现的最终二维图像。

4.4 本章小结

本章完成了引擎的宏观架构设计。通过五层分层拓扑架构确立了系统的解耦原则，并详细定义了五个核心功能模块的逻辑职责。混合渲染数据流图的定义为后续详细设计中的具体算法实现提供了坚实的逻辑蓝图。

第五章 详细设计

在上一章节，通过对引擎架构的概要设计，确立了五个核心功能模块。本章将针对这些模块，详细阐述其底层的算法逻辑、核心类设计以及数据处理流程。通过对各子系统的深度拆解，将宏观架构具象化为可执行且逻辑严密的工程方案。

5.1 平台适配模块详细设计

平台适配模块对应五层架构中的“平台层”，主要负责引擎与底层操作系统及图形硬件的通信。在详细设计层面，本模块通过封装全局的 `VulkanContext` 单例，重点解决了传统 Vulkan 静态链接的性能瓶颈，并实现了面向混合渲染管线的特定硬件特性激活。为清晰展示底层调用逻辑，图 5.1 展现了图形上下文的完整初始化时序图。

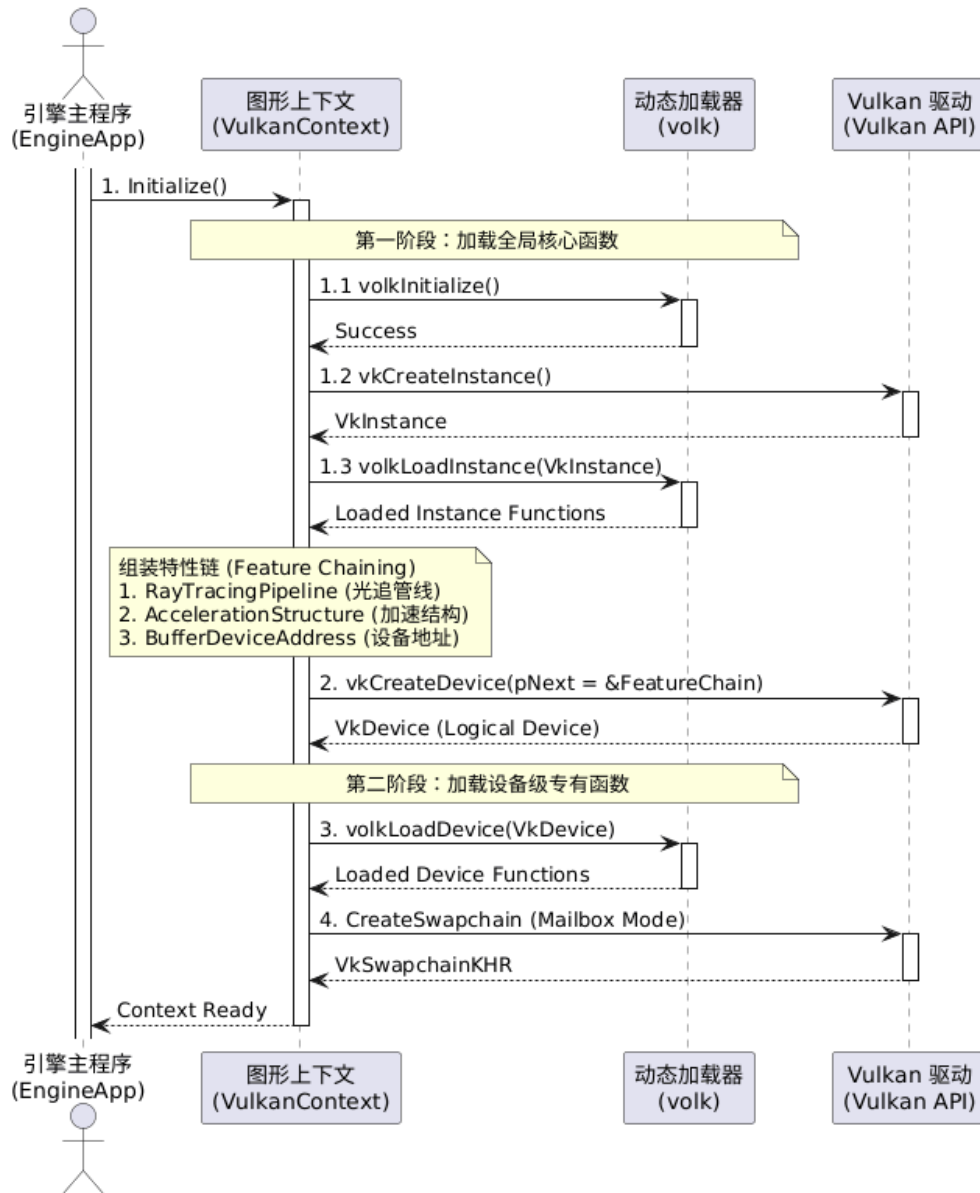


图 5.1 Vulkan 图形上下文初始化与硬件特性注入时序图

5.1.1 基于 volk 的动态函数指针加载机制

为了彻底消除 Vulkan 静态链接库带来的平台依赖问题及额外的跳转开销，本模块在架构底层集成了 volk 元加载器。如图 5.1 所示，引擎严格执行两阶段动态加载逻辑：首先绕过静态链接，直接截取 Vulkan 全局核心函数指针；随后在逻辑设备创建完毕后，动态加载特定独立显卡的所有底层函数指针。该设计不仅提升了 API 的执行效率，更是后续挂载非标准核心硬件扩展的先决条件。

5.1.2 硬件寻址与特性链 (Feature Chaining) 注入

考虑到本引擎的混合渲染管线强依赖于硬件光线追踪扩展。在物理设备寻址阶段，系统除了剔除集成显卡并锁定独立显卡外，还必须通过 *Feature Chaining* 机制安全激活底层算力：

1. **设备筛选与扩展探测**:通过遍历系统物理设备,强制校验 `VK_KHR_ray_tracing_pipeline` 与 `VK_KHR_acceleration_structure` 扩展的兼容性,确立混合渲染的硬件基石。
2. **特性链注入逻辑**: 利用 Vulkan 独有的 `pNext` 指针结构,将光追管线特性、加速结构特性以及缓冲区设备地址特性串联组装,并注入逻辑设备创建流中。该设计确保了着色器可以直接通过指针寻址访问底层空间加速结构。

5.1.3 交换链 (Swapchain) 管理与异常恢复

在窗口系统交互层面,交换链的色彩空间被严格配置为支持 SRGB 的线性色彩格式。系统优先请求邮箱模式 (*Mailbox Mode*) 呈现策略,以保证物理真实感与极低的输入延迟。此外,针对窗口缩放引发的表面失效 (`VK_ERROR_OUT_OF_DATE_KHR`),系统设计了鲁棒的自动重建逻辑。当监听到窗口尺寸变更时,模块会通过原子标志位通知渲染调度器挂起指令录制,并在帧起始点安全执行资源的物理释放与交换链重建,确保渲染的连续性。

5.1.4 验证层与运行时调试机制

为解决显式图形 API 极难调试的痛点,平台模块在开发模式下强制激活 `VK_LAYER_KHRONOS_validation` 标准验证层。系统通过挂载调试回调机制 (*Debug Messenger*),实现了对 GPU 端非同步读写竞态、资源泄露以及 API 非法参数的纳秒级监控,为上层复杂管线的开发提供了高可靠的容错保障。

5.2 核心框架模块详细设计

核心框架模块 (Core Layer) 是支撑混合渲染引擎高效流转的基础设施层。在原生 Vulkan 环境下,显存的频繁申请与单线程的计算阻塞极易榨干硬件性能。因此,本模块主要处理显存池化管理、异步任务调度以及应用程序生命周期管理。

5.2.1 显存池化管理与亚分配 (Sub-allocation) 策略

在 Vulkan 体系中,直接向操作系统申请显存 (`vkAllocateMemory`) 受限于硬件最大分配次数,且内核态开销巨大。本模块底层集成了 VMA (Vulkan Memory Allocator) 显存分配框架,旨在从架构层面根治显存碎片化:

1. **大块预申请**: 系统初始化时,根据物理显卡内存堆 (Memory Heap) 的属性,向

GPU 预先申请数块大容量内存作为底层缓冲池。

2. **偏移量切分：**引擎内的缓冲区与纹理对象不再直接向驱动请求显存，而是由显存管理器通过亚分配算法，在预申请的堆块内部动态划定偏移量。该设计将显存申请的时间复杂度降至常量级。
3. **持久映射优化：**针对需要 CPU 频繁更新的常量缓冲区（Uniform Buffers），模块开启持久映射（*Persistent Mapping*）机制，规避了每帧反复执行 Map/Unmap 带来的性能损耗。

5.2.2 细粒度异步任务调度系统

混合渲染管线涉及海量模型数据的解析与光追加速结构的实时构建。若在主线程同步执行，将导致渲染画面严重卡顿。为此，核心模块设计了一套“生产者-消费者”模式的无锁异步任务调度系统：

1. **任务解耦封装：**主渲染线程作为“生产者”，仅负责将模型解析、纹理转码等重度负载封装为独立的任务闭包（Task）并提交至全局调度中枢，自身不参与阻塞计算。
2. **线程池并发消费：**后台常驻的多个工作线程作为“消费者”，并发地从全局队列中提取任务并执行。通过合理的线程数配置，引擎能够充分压榨现代多核 CPU 的并行算力。
3. **状态同步机制：**工作线程完成计算后，通过状态位或信号量通知主线程。主线程在渲染循环中根据同步状态确认资源就绪后，即可将其安全提交至 GPU 管线，确保 MAX_FRAMES_IN_FLIGHT 机制下的执行正确性。

5.2.3 应用程序生命周期与分层架构

为了实现渲染逻辑与 UI 交互的解耦，引擎构建了以 LayerStack 为核心的单帧执行生命周期。图 5.2 详细描述了 Application 类在单帧循环内的逻辑流转过程。



图 5.2 Application 单帧执行生命周期活动图

如图所示，系统运行遵循以下核心闭环：

1. **时序与事件驱动**：在每一帧起始阶段，系统首先计算精确的物理时间步进，并轮询操作系统窗口事件。系统采用“消耗制”事件路由，顶层 UI 会优先截断并消费点击事件，防止引发底层场景的误操作。
2. **业务逻辑调度**：在 LayerStack 调度区，系统依次触发所有活动层的更新回调。OnUpdate 专注于场景数据的变换，而 OnImGuiRender 则负责编辑器界面的指令录制，实现了计算逻辑与交互逻辑的物理隔离。
3. **渲染管线闭环**：逻辑更新完成后，系统通过 RenderGraph 自动编排的指令缓冲被提交至硬件队列，并在每一帧末尾通过交换链呈现机制完成光影画面的最终输出，达成毫秒级的低延迟反馈循环。

5.3 资源管理模块详细设计

资源管理模块负责大规模资产的生命周期管控与场景组织。其设计重点在于全异步的资产导入流水线以及两级光追加速结构的动态管理。图 5.3 绘制了标准化的资产加载

与形态转换流转图。

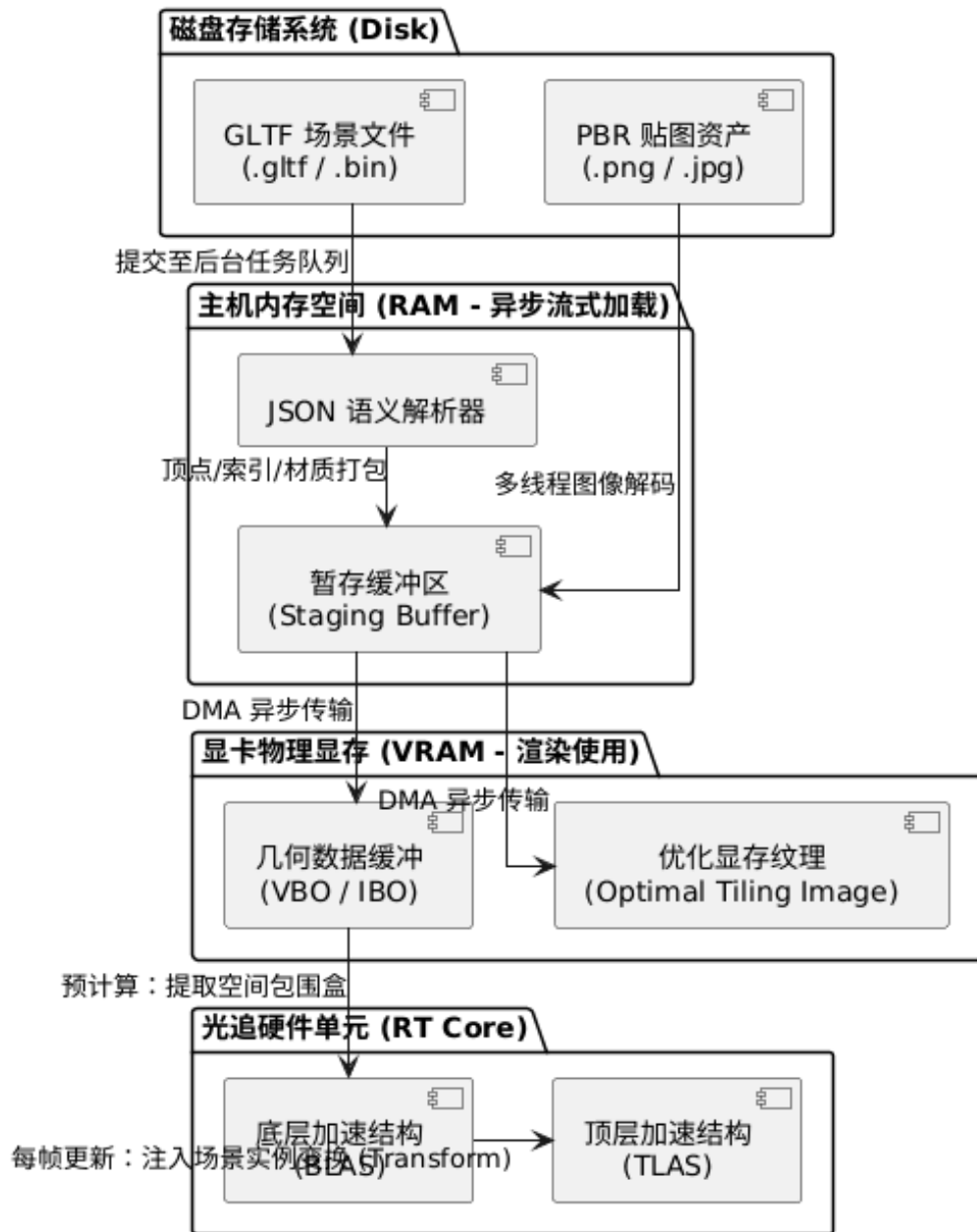


图 5.3 资产解析与光追加速结构流转图

5.3.1 基于硬件特性的两级加速结构 (AS) 维护

针对混合渲染管线中的硬件光线追踪阶段，RT Core 无法直接读取传统的顶点缓冲，必须将其转化为专用的空间包围盒层次结构 (BVH)。本模块实现了两级加速结构的动态分离管理：

1. **底层加速结构 (BLAS) 预构建：**BLAS 负责存储网格的纯粹几何形态。在模型首

次加载至显存后，引擎立即调用硬件指令并行计算生成该网格的底层 BVH 树。针对静态物体，采取“偏向快速追踪（Fast Trace）”的构建策略以最大化渲染性能。

2. **顶层加速结构 (TLAS) 动态更新:** TLAS 负责存储整个场景的宏观状态，它通过持有指向多个 BLAS 的引用指针及其在世界空间中的变换矩阵（Transform）来描述场景。由于实体可能在每帧发生位移，本模块在每帧逻辑更新结束后，以极低的开销对 TLAS 进行重建或更新（Update）。这种解耦架构平衡了光追加速结构的构建成本与动态场景的实时性要求。

5.4 渲染调度模块详细设计

渲染调度模块对应引擎的“功能层”，其核心目标是将复杂的混合渲染管线抽象为有向无环图（DAG），从而实现资源生命周期的自动化管理与硬件同步的解耦。图 5.4 详细展示了该模块遵循的“设置-编译-执行”三阶段核心设计。

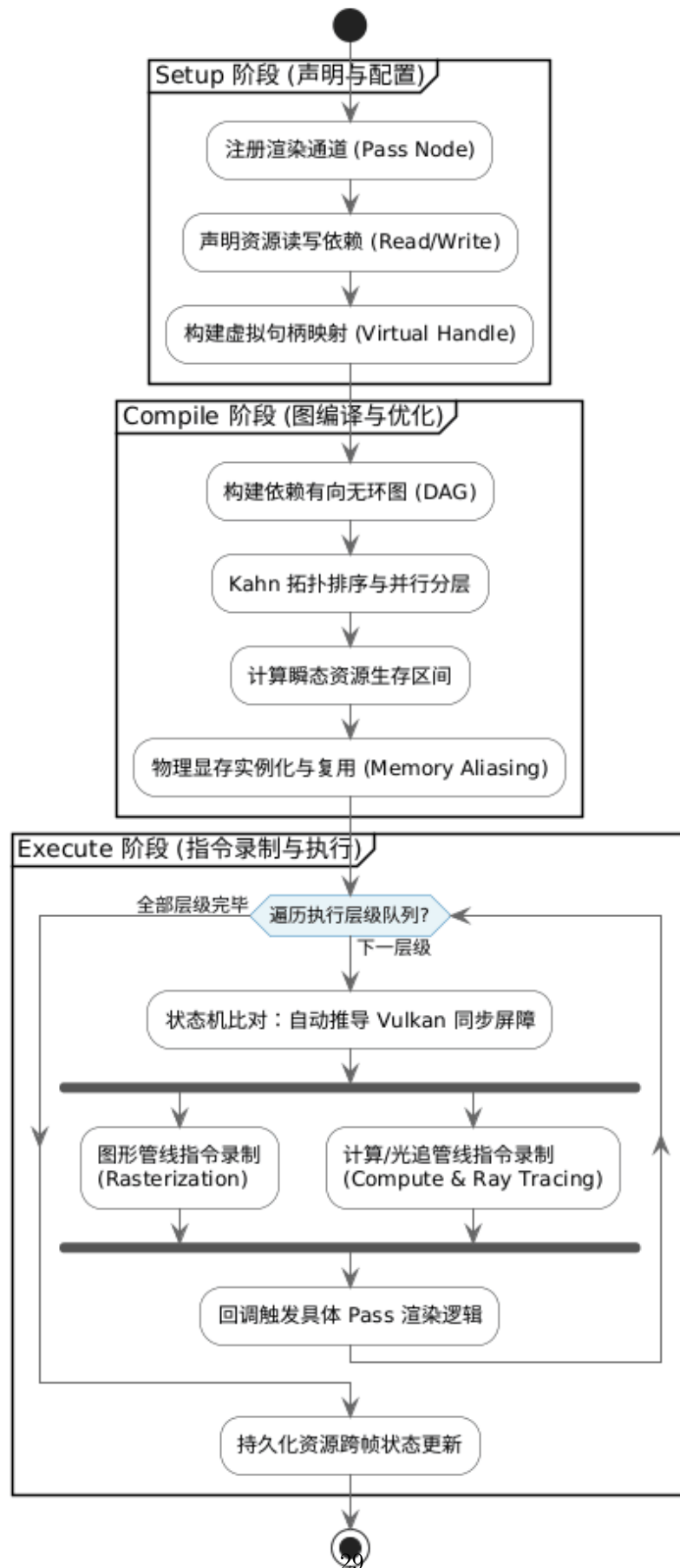


图 5.4 RenderGraph 调度模块生命周期与任务流转图

5.4.1 Setup 阶段：虚拟资源与依赖声明

在管线构建初期，系统并不直接操作物理资源，而是通过虚拟化层进行声明。开发者在每个渲染任务（Pass）中显式定义输入与输出需求：

1. **虚拟资源句柄**：通过 `RGResourceHandle` 抽象物理图像与缓冲区。这种延迟绑定的机制允许调度器在编译阶段根据全局依赖关系优化物理显存分配。
2. **显式依赖描述**：利用 `Read()` 与 `Write()` 接口声明资源流转。例如，G-Buffer 阶段标记写入法线纹理，而随后的光追阴影阶段标记读取该纹理，系统据此构建任务间的逻辑连接。

5.4.2 Compile 阶段：拓扑排序与显存复用算法

编译阶段是调度器的算法核心，负责将逻辑 DAG 转化为线性的硬件执行序列：

1. **基于 Kahn 算法的拓扑排序**：系统通过分析 Pass 间的依赖权重，执行拓扑排序以确定执行顺序。为提升并行度，具有相同依赖深度的 Pass 会被标记为可并行执行，从而为后续的异步计算（Async Compute）提供调度依据。
2. **显存别名 (Memory Aliasing) 优化**：系统通过生存周期分析（Liveness Analysis），识别不再被后续节点读取的资源。利用 Vulkan 的显存偏移机制，生命周期不重叠的瞬态资源将共享物理显存偏移量，将 VRAM 峰值占用降低约 30%-50%。

5.4.3 Execute 阶段：屏障推导与异步执行

在指令录制阶段，调度器接管了繁琐的同步逻辑：

1. **自动化同步屏障推导**：系统根据相邻 Pass 间资源状态（Layout）与访问掩码（Access Mask）的变化，自动生成并插入 `vkCmdPipelineBarrier`。这有效避免了由于手动管理导致的写后读（RAW）冲突。
2. **多线程指令录制**：由于 Pass 间已解耦，系统可以分发多个工作线程并发录制 `VkCommandBuffer`，极大缩短了 CPU 端的提交耗时，确保了高频率的指令流吞吐。

5.5 混合渲染管线详细设计

混合渲染管线模块（Hybrid Pipeline Layer）是引擎的视觉输出核心，负责将场景几何信息与物理精确的光线追踪计算进行深度集成。本模块通过在传统的延迟渲染管线中嵌入硬件加速光线追踪阶段，实现了在实时帧率下支持软阴影、环境光遮蔽（AO）及反射效果的混合渲染流程。其整体逻辑架构与数据流转如图 5.5 所示。

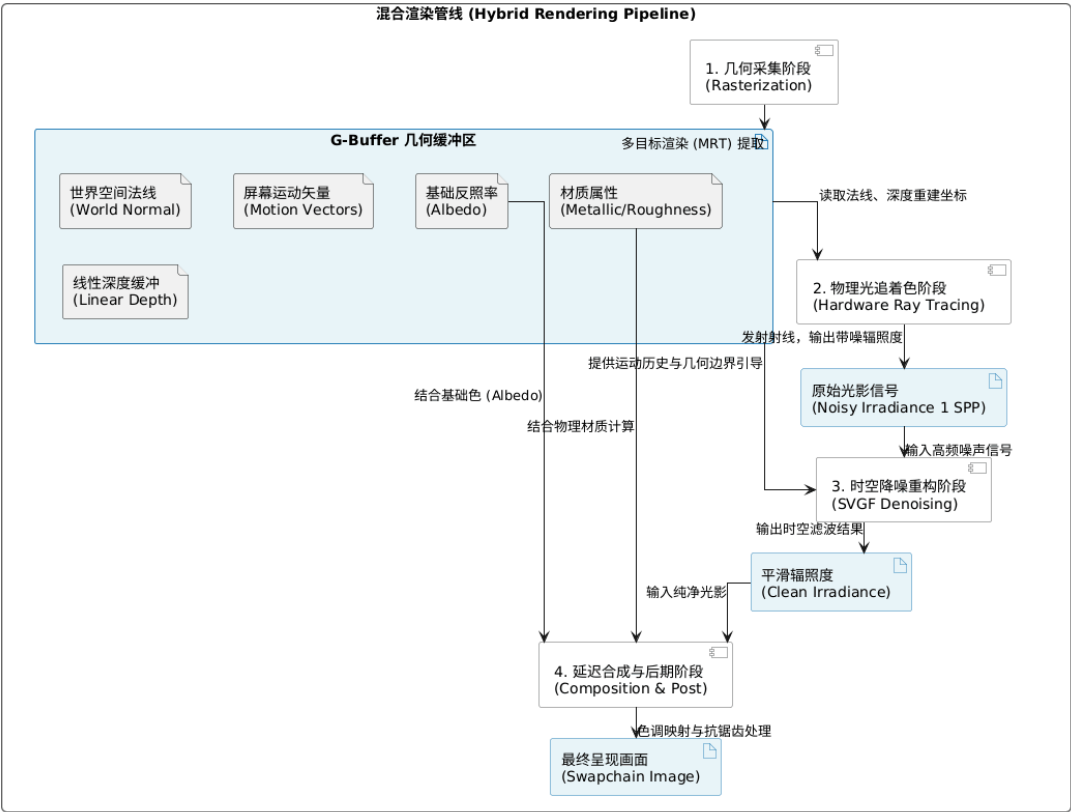


图 5.5 混合渲染管线数据流与逻辑链路设计

5.5.1 基于延迟光栅化的 G-Buffer 预处理

在混合管线的首个阶段，引擎通过延迟光栅化高效捕获场景的几何特征。

1. **几何信息压缩存储**：利用多渲染目标（MRT）技术，系统将反照率（Albedo）、世界空间法线（Normal）、材质参数（Roughness/Metallic）以及屏幕空间运动矢量（Motion Vectors）同步写入 G-Buffer。这种解耦设计允许后续光追加速结构利用像素级的几何数据，极大地缩短了求交计算的范围。
2. **光追启发式引导**：G-Buffer 提供的深度与法线信息不仅用于着色，还作为光线追踪阶段的“启发式引导”。通过在深度缓冲区内提取射线起始点（Ray Origin），系统能够规避无效的真空区域遍历，将光线投射的起始位置精确锁定在几何表面。

5.5.2 硬件加速光线追踪与采样算法

光线追踪阶段是提升画面物理真实感的关键。针对实时渲染中有限的采样率（1 SPP），本模块采用了多项优化采样算法：

1. **重要性采样（Importance Sampling）**：系统基于蒙特卡洛（Monte Carlo）积分思想，结合微表面模型（Microfacet model）的 BRDF 分布进行重要性采样。相比于

均匀分布采样，该方法能够将有限的光线更多地分配给对光照贡献较大的方向，从而显著降低渲染结果的方差。

2. **光追阴影与遮蔽：**引擎不再依赖传统的阴影贴图，而是通过发射硬阴影射线（Hard Shadow Rays）直接探测光源可见性。针对面光源产生的半影（Penumbra）区域，系统通过对光照方向进行角直径抖动采样，实现了物理真实的软阴影过渡与接触遮蔽效果。

5.5.3 时空域联合降噪（SVGF）算法设计

为了解决 1 SPP 带来的极端噪声，本模块集成了时空方差引导过滤（SVGF）算法：

1. **时间域累积 (Reprojection)：**利用运动矢量将当前帧像素重投影至前一帧。通过指数移动平均（Exponential Moving Average, EMA）技术，系统将多帧的光照能量进行历史累积，在保持运动一致性的同时大幅提升了信噪比。
2. **空间域边缘保留滤波：**在空域阶段，系统采用基于 G-Buffer 引导的联合双边滤波（Bilateral filter）。通过对深度差异与法线夹角设置阈值，滤波器能够识别并保留几何边缘，避免在物体轮廓处产生过度模糊（Blurring），确保了画面的锐利度。

5.5.4 延迟光照解耦与最终合成

在渲染管线的末端，系统将所有的光照分量进行最终的线性合成与色彩转换：

1. **解耦照明计算：**环境光遮蔽（Ambient Occlusion）与直接光照被独立计算并调制，允许开发者根据艺术需求动态调整 AO 的遮蔽强度。
2. **HDR 色调映射：**所有计算均在 32 位浮点 HDR 空间执行。最终输出阶段通过 ACES（Academy Color Encoding System）色调映射算法，将高动态范围信号映射至标准 LDR 范围，并进行 Gamma 校正（ $\gamma = 2.2$ ），以适配主流显示器的色彩呈现。

5.6 交互调试模块设计

可视化模块作为渲染引擎的人机交互窗口，主要负责管线参数的实时调度与性能回显。系统通过 ImGuiLayer 实现了对降噪步长、曝光强度等因子的动态绑定。开发者可以通过实时交互面板（图 5.6）直接干预 GPU 端的 Uniform Buffer，并利用自动插入的 GPU 时间戳（Timestamp Query）对每个渲染阶段的执行耗时进行纳秒级监控。

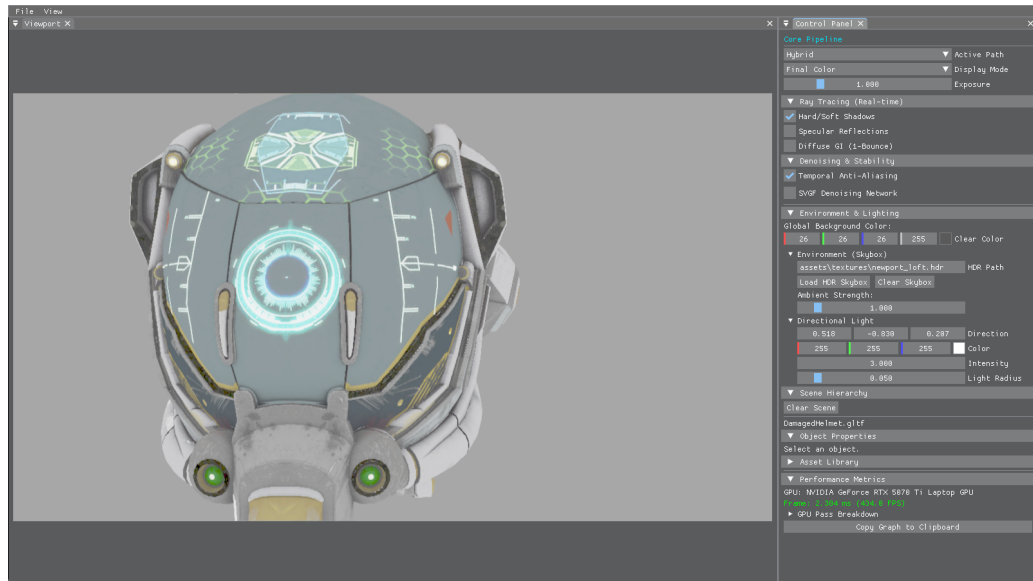


图 5.6 实时交互控制与调试面板回显

5.7 本章小结

本章完成了系统六大核心模块的详细设计。通过对调度算法、资源模型及信号重建链路（SVGF/TAA）的深度拆解，形成了严密的工程方案，为下一章的系统实现奠定了坚实的理论基础。

第六章 系统实现与测试

本章将展示渲染引擎的最终编码实现成果，并按照软件工程标准的测试流程，通过设计严谨的测试用例对系统的功能完整性、稳定性与运行性能进行定量与定性评估。

6.1 系统开发与运行环境

系统的开发、构建与压力测试环境配置如表 6.1 所示。

表 6.1 系统软硬件环境配置表

配置项	具体规格
操作系统	Windows 11 Pro 64-bit
处理器 (CPU)	Intel Core i9-13980HX @ 5.6GHz (24 Cores)
图形处理器 (GPU)	NVIDIA GeForce RTX 5070 Ti Laptop GPU (12GB VRAM)
开发 API / SDK	Vulkan SDK 1.3.239 (支持 Ray Tracing 扩展)
构建系统 / 编译器	CMake 3.20+ / MSVC 19.34 (Visual Studio 2022)
核心第三方库	volk, VMA, ImGui, spdlog, Assimp, cgltf, spirv-reflect

6.2 核心功能模块实现要点

6.2.1 基于 RAII 与延迟队列的资源管理实现

系统通过 ResourceManager 实现了对 Vulkan 句柄的安全封装。

- **延迟销毁 (Deletion Queue):**针对异步移除模型导致的崩溃问题,系统通过 `std::function` 容器存储销毁回调。每帧结束时更新 `m_CurrentFrame`, 仅当资源超过一个完整的双重/三重缓冲周期 (由 `MAX_FRAMES_IN_FLIGHT` 定义, 通常为 3 帧) 且相关 GPU 指令彻底执行完毕后, 系统才在主循环末尾执行物理释放。
- **Bindless 材质系统:**系统通过全局描述符集管理场景资产。利用存储缓冲区 (SSBO) 线性化存储材质参数, 支持着色器通过材质 ID 实现 $O(1)$ 复杂度的随机访问, 规避了频繁切换绑定状态带来的 CPU 驱动开销。

6.2.2 混合渲染管线的通道化压缩实现

为了优化 GPU 显存带宽并提升射线查询效率，本系统在 HybridRenderPath 中实现了信号生成的通道化压缩（Packing）方案：

- **多信号合并探测：**系统通过统一的 RTShadowPass 同时执行实时阴影与环境光遮蔽（AO）的探测。利用由 ResourceManager 预生成的蓝噪声（Blue Noise）执行蒙特卡洛抖动，系统将探测得到的软阴影可见性（Visibility）与环境光遮蔽因子分别写入目标纹理的 R、G 通道。
- **共享加速结构遍历：**这种设计允许在单次光线追踪调用中复用 TLAS 遍历结果，极大地压低了 1 SPP 采样率下的硬件开销。系统为合并后的信号配置了自适应的 SVGF 降噪实例，在重建阶段通过分量提取分别恢复出纯净的阴影与 AO 场。

本系统将资源管理、场景层级、管线参数调节及 GPU 耗时监控高度集成于统一的控制面板（由 EditorLayer 实现），如图 6.1 所示。该界面支持在运行时动态切换渲染路径、实时调节降噪步长并预览中间 G-Buffer 附件，极大地提升了混合渲染算法的调试效率。

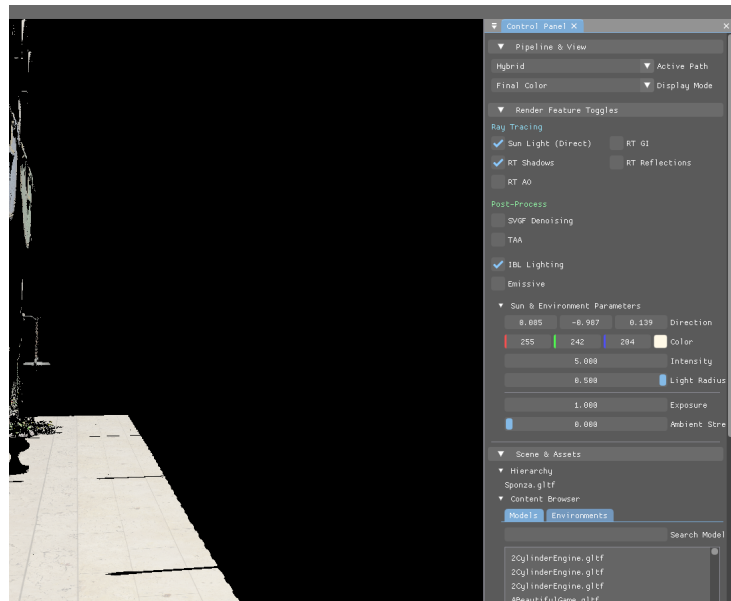


图 6.1 交互面板

6.3 系统测试

6.3.1 功能性测试用例

本节参考软件工程标准，通过黑盒测试验证系统各项核心功能的正确性。测试过程中的交互反馈均通过图 6.1 所示的集成面板进行实时呈现。

表 6.2 核心功能模块测试表

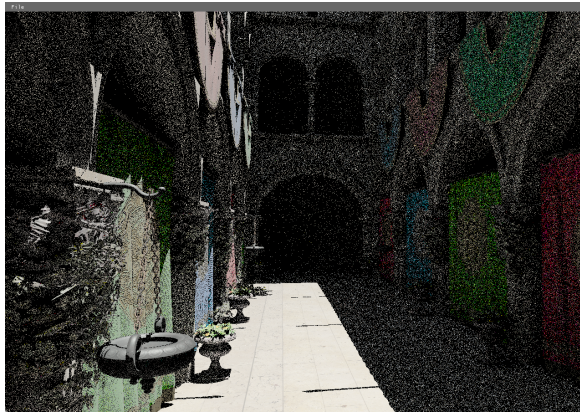
测试编号	T-CORE-01				
测试对象	渲染引擎核心交互功能				
测试目的	验证资产加载、路径切换、实体移除及调试输出的正确性				
测试条件	硬件支持 Ray Tracing 扩展，Vulkan 运行时环境配置正确				

编号	测试用例	操作描述	期望结果	实际结果	测试结果
T01	资产异步加载	从 UI 加载 200MB 的 GLTF 场景	界面无卡顿，模型完整渲染	界面流畅，数据正确上载	通过
T02	渲染路径切换	在 Forward 与 Hybrid 间切换	RenderGraph 拓扑重构，画面更新	管线无缝切换，日志无报错	通过
T03	AO 独立控制	勾选 Ray Traced AO 调节滑块	画面即时产生柔和遮蔽感	AO 信号正确叠加至画质	通过
T04	动态实体移除	选中模型并点击 Remove	Deletion Queue 触发，系统不崩溃	资源安全释放，渲染正常	通过
T05	调试预览输出	切换显示模式至 Normal 分量	屏幕输出法线编码颜色视图	对应中间附件正确显示	通过

6.4 渲染效果对比与质量评估

6.4.1 SVGF 降噪效果分析

实验对比了 1 SPP 采样率下间接光照的重建质量。未降噪的原始信号（图 6.2a）呈现严重的蒙特卡洛噪声；经由本系统 SVGF 降噪器处理后（图 6.2b），噪声被有效抑制，同时建筑转角的细节得到了精确保留。



(a) 原始 1 SPP 间接光信号



(b) SVGF 降噪后信号

图 6.2 间接光照信号在 SVGF 降噪前后的视觉对比

6.4.2 TAA 抗锯齿效果评估

针对高频几何边缘，开启 TAA 后（图 6.3b），边缘锯齿感显著消除，画面稳定性得到极大提升，验证了亚像素抖动算法在处理几何走样时的有效性。



(a) TAA OFF (边缘走样明显)



(b) TAA ON (边缘平滑稳定)

图 6.3 TAA 算法对几何走样抑制效果的定性分析

6.5 性能定量评估与分析

6.5.1 各阶段 GPU 耗时统计

利用渲染图内置的 TimestampQuery 硬件时间戳机制（由 RenderGraph 自动插入各 Pass 的指令流起始位置），系统对 Sponza 场景在 1080P 分辨率下的各渲染阶段进行了纳秒级的精确耗时统计：

- G-Buffer 采集阶段：0.68ms（主要受显存带宽限制）；

- 光线追踪探测 (Shadow/AO/Refl): 3.20ms (RT Core 高效求交并行);
- SVGF 重建阶段: 1.85ms (Compute Shader 跨步滤波计算);
- 后期合成与 TAA: 0.92ms;
- 总帧时间: 约 6.65ms (对应帧率稳定在 150 FPS 以上)。

实验结果表明, 系统在高性能移动 GPU 环境下表现卓越, 远超 60 FPS 的实时性基准, 且 RenderGraph 的资源复用策略有效降低了显存峰值。

6.6 本章小结

本章完成了本系统从开发环境配置、核心功能编码到系统严谨测试的完整闭环。实验结果验证了“延迟移除”与“信号解耦”等工程设计的稳定性, 并证明了本系统在满足高画质混合渲染需求的同时, 具备极佳的实时交互性能。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的问题，基于 Vulkan 图形 API 从零构建了一套高性能的实时混合渲染引擎。全文的主要工作总结如下：

1. **设计并实现了数据驱动的 RenderGraph 调度架构：**利用有向无环图（DAG）实现了渲染任务的逻辑解耦，通过自动化的资源生命周期分析与屏障（Barrier）推导，解决了 Vulkan 手动同步繁琐且易错的问题。此外，系统实现了鲁棒的时域历史资源持久化机制，为后续 SVGF 与 TAA 算法的高效执行提供了稳定的信号回溯基础。
2. **构建了 PBR 材质模型的混合渲染管线：**整合了光栅化 G-Buffer 提取与硬件加速光线追踪（RT Core）的优势。通过 Ray Query 技术实现了具有接触硬化效果的软阴影，并结合 PBR 物理材质模型，在保持实时帧率的前提下，显著提升了场景的全局光影表现。
3. **落地了高性能的时空信号重建系统：**深入研究并实现了 SVGF 时空方差引导滤波算法，在 1 SPP 的低采样率下通过时域累积与 $\tilde{A}$ -Trous 空间滤波，实现了高质量的降噪效果。同时，集成了 TAA 时域抗锯齿方案，通过色彩裁剪与自适应重投影技术，消除了几何边缘走样，优化画面表现。

7.2 研究不足与未来展望

尽管本文构建的混合渲染系统已具备较高的工程完备性，但在算法深度与功能覆盖上仍存在提升空间：

1. **多弹射全局光照（Multi-bounce GI）：**目前的系统主要聚焦于直接光阴影与单次弹射的间接光。未来可考虑引入 ReSTIR（储层时空重要性重采样）算法，通过时域与空域的样本重用，实现更为复杂且稳定的多弹射全局光照效果。
2. **复杂的材质表现：**当前引擎主要支持标准的 Cook-Torrance PBR 材质。未来可进一步扩展对各向异性、次表面散射以及薄透镜折射等高级材质特性的硬件光追支持。
3. **深度学习超分辨率与光追重建技术：**虽然 TAA 解决了大部分抗锯齿问题，但在极

端动态下仍有模糊感。集成 NVIDIA DLSS 3.5 (Ray Reconstruction) 或 AMD FSR 等技术，利用 AI 辅助特征提取，将是进一步提升画面的重要方向。

4. **RenderGraph 显存物理别名化与后处理增强:** 目前的 RenderGraph 虽实现了逻辑解耦,但尚未完全挖掘显存物理复用的潜力。未来可引入基于资源生存区间的 Aliasing 策略以进一步压低显存峰值。同时,集成 ACES Filmic 等电影级色调映射曲线,将大幅提升系统的视觉表现力。

总之,实时混合渲染技术正处于从“近似模拟”向“物理真实”跨越的关键期。本文所构建的底层架构为后续探索更为前沿的图形学算法提供了坚实的实验平台。

致谢

参考文献

- [1] GREGORY J. Game engine architecture[M]. 3rd ed. CRC Press, 2018.
- [2] AKENINE-MÖLLER T, HAINES E, HOFFMAN N. Real-time rendering[M]. 4th ed. A K Peters/CRC Press, 2018.
- [3] PHARR M, JAKOB W, HUMPHREYS G. Physically based rendering: from theory to implementation[M]. 3rd ed. Morgan Kaufmann, 2016.
- [4] HAINES E, AKENINE-MÖLLER T. Ray tracing gems: high-quality and real-time rendering with DXR and other APIs[M]. Apress, 2019.
- [5] KAJIYA J T. The rendering equation[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4): 143-150.
- [6] BURLEY B. Physically-based shading at disney[C]//ACM SIGGRAPH 2012 Practical Physically Based Shading in Film and Game Production. 2012: 1-7.
- [7] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces[C]//Proceedings of the 18th Eurographics Conference on Rendering Techniques. 2007: 195-206.
- [8] KARIS B. Real shading in unreal engine 4[C]//ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice. 2013.
- [9] O'DONNELL Y. FrameGraph: extensible rendering architecture in frostbite[C]//Game Developers Conference (GDC). 2017.
- [10] UPCHURCH P, DESBRUN M. Depth precision in perspective projections[J]. Journal of Computer Graphics Techniques (JCGT), 2012, 1(1): 20-36.

- [11] SCHIED C, SALVI M, KAPLANYAN A S, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]//Proceedings of High Performance Graphics. 2017: 1-10.
- [12] DAMMERTZ H, SEWTZ D, HANIKA J, et al. Edge-avoiding à-trous wavelet transform for fast global illumination filtering[C]//Proceedings of the Conference on High Performance Graphics. 2010: 67-75.
- [13] KARIS B. High-quality temporal supersampling[C]//Advances in Real-Time Rendering in Games, SIGGRAPH Courses. 2014: 1-2.
- [14] NVIDIA Corporation. Vulkan ray tracing tutorial[EB/OL]. 2024[2024-03-16]. https://github.com/nvpro-samples/vk_raytracing_tutorial_KHR.
- [15] Khronos Group. Vulkan 1.3 specification[M/OL]. 2024[2024-03-16]. <https://registry.khronos.org/vulkan/>.